Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «КиСБ»
Тема: «Блочные шифры и классические режимы шифрования»

Выполнил:
Студент 4-го курса
Группы АС-65
Гуща И.В.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: программная реализация симметричного блочного шифра (AES-128 или СТБ 34.101.31 «БелТ») и классических режимов работы (ECB, CBC, CFB, OFB, CTR) с поддержкой алгоритма дополнения блоков (Padding)

Задание.

В лабораторной работе требуется реализовать указанный блочный шифр и продемонстрировать его работу в двух режимах шифрования: обязательном базовом режиме ECB и дополнительном режиме согласно варианту.

Вариант 3:

3	AES-128	**ECB + CFB**	Демонстрация использования исключительно функции EK для расшифрования в режиме CFB.

AES:

Исходный блок — 16 байт

AddRoundKey

XOR с начальным раундовым ключом

Раунд 1 -9

SubBytes → ShiftRows → MixColumns → AddRoundKey

Раунд 10

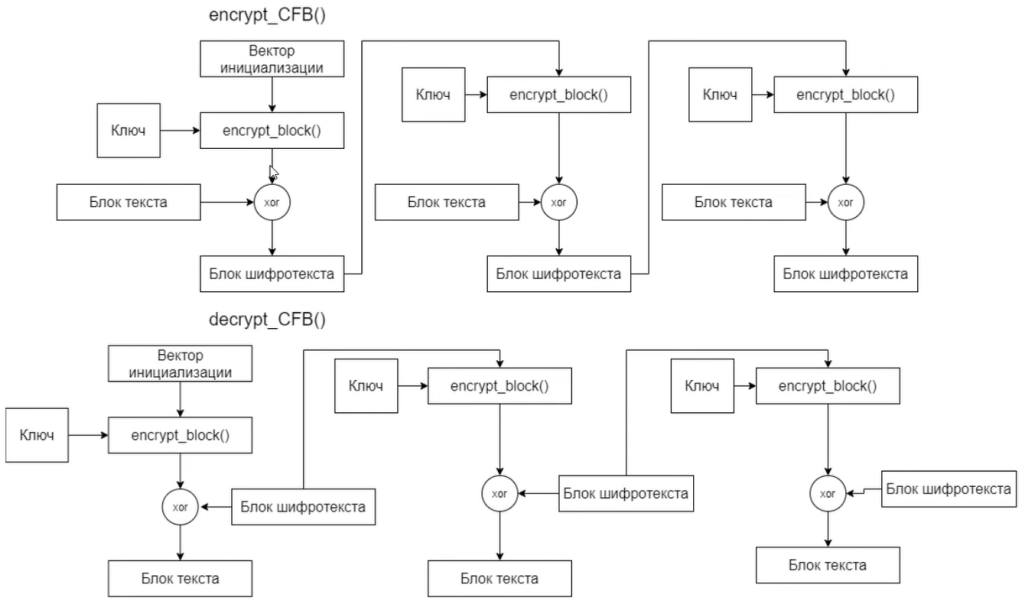
SubBytes → ShiftRows → AddRoundKey

Без MixColumns

Шифротекст — 16 байт

ECB режим шифрует блоки независимо один от одного, что делает его уязвимым для статистического анализа.

CFB использует вектор инициализации, который после шифровки ксориться с исходным текстом. Далее шифротекст передается на следующий блок, кодируется и также ксориться с текстом. При расшифровке используется шифрование вектора инициализации и ксор с блоком шифротекста.



Основные функции encrypt_ECB, decrypt_ECB, encrypt_CFB, decrypt_CFB. Они отвечают за выбранный режим шифрования, и из них вызывается функция с алгоритмом шифрования.

```
def encrypt_ECB():
    filename = input("Имя исходного .txt файла: ").strip()
    key = read_key()
    with open(filename, "rb") as f:
        data = padding(f.read())
    result = b"".join(encrypt(data[i:i+16], key) for i in range(0, len(data),
16))
    out_name = filename + ".ecb"
    with open(out_name, "wb") as f:
        f.write(result)
    print("Зашифровано. Результат:", out_name)

def decrypt_ECB():
    filename = input("Имя зашифрованного файла (.ecb): ").strip()
    key = read_key()
    with open(filename, "rb") as f:
        data = f.read()
    if not data or len(data) % 16:
        raise ValueError("Размер ECB-файла должен быть кратен 16 байтам.")
    result = b"".join(decrypt(data[i:i+16], key) for i in range(0, len(data),
16))
    result = unpadding(result)
    out_name = filename + ".decrypted.txt"
    with open(out_name, "wb") as f:
        f.write(result)
    print("Расшифровано. Результат:", out_name)

def encrypt_CFB():
    filename = input("Имя исходного .txt файла: ").strip()
    key = read_key()
    iv = read_iv()
    with open(filename, "rb") as f:
        data = padding(f.read())
    result = bytearray()
    feedback = iv
    for i in range(0, len(data), 16):
        block = data[i:i+16]
        gamma = encrypt(feedback, key)
        cipher_block = xor_bytes(block, gamma)
        result.extend(cipher_block)
        feedback = cipher_block
    out_name = filename + ".cfb"
    with open(out_name, "wb") as f:
        f.write(iv + result)                    # IV сохраняется в начале файла
    print("Зашифровано. Результат:", out_name)

def decrypt_CFB():
    filename = input("Имя зашифрованного файла (.cfb): ").strip()
    key = read_key()
    with open(filename, "rb") as f:
        content = f.read()
    if len(content) < 32 or (len(content) - 16) % 16:
        raise ValueError("Неверный формат CFB-файла.")
    feedback = content[:16]                     # IV
    data = content[16:]
    result = bytearray()
    for i in range(0, len(data), 16):
        cipher_block = data[i:i+16]
        gamma = encrypt(feedback, key)
```

```
            plain_block = xor_bytes(cipher_block, gamma)
            result.extend(plain_block)
            feedback = cipher_block
        result = unpadding(bytes(result))
        out_name = filename + ".decrypted.txt"
        with open(out_name, "wb") as f:
            f.write(result)
        print("Расшифровано. Результат:", out_name)


def encrypt(block, key):
    #Шифрует ровно один 16-байтовый блок AES-128.
    if len(block) != 16:
        raise ValueError("Блок должен иметь длину 16 байт.")
    keys = expand_key(key)
    state = add_round_key(list(block), keys[0])
    for r in range(1, 10):
        state = sub_bytes(state)
        state = shift_rows(state)
        state = mix_columns(state)
        state = add_round_key(state, keys[r])
    state = sub_bytes(state)
    state = shift_rows(state)
    state = add_round_key(state, keys[10])
    return bytes(state)

def decrypt(block, key):
    #Расшифровывает ровно один 16-байтовый блок AES-128.
    if len(block) != 16:
        raise ValueError("Блок должен иметь длину 16 байт.")
    keys = expand_key(key)
    state = add_round_key(list(block), keys[10])
    for r in range(9, 0, -1):
        state = inv_shift_rows(state)
        state = inv_sub_bytes(state)
        state = add_round_key(state, keys[r])
        state = inv_mix_columns(state)
    state = inv_shift_rows(state)
    state = inv_sub_bytes(state)
    state = add_round_key(state, keys[0])
    return bytes(state)
def padding(data):
    #PKCS#7: добавляет от 1 до 16 байтов.
    count = 16 - (len(data) % 16)
    return data + bytes([count] * count)

def unpadding(data):
    #Проверяет и удаляет PKCS#7.
    if not data or len(data) % 16 != 0:
        raise ValueError("Неверная длина данных для снятия padding.")
    count = data[-1]
    if count < 1 or count > 16 or data[-count:] != bytes([count] * count):
        raise ValueError("Неверный padding")
    return data[:-count]
```

Результат работы программы

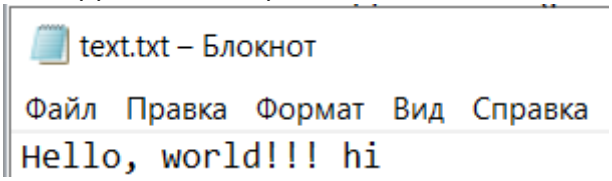
AES-128: выберите режим
1 — Шифрование ECB
2 — Расшифрование ECB
3 — Шифрование CFB
4 — Расшифрование CFB
0 — Выход
Ваш выбор: 3

Имя исходного .txt файла: text.txt
Введите ключ (ровно 16 символов, например abcdefgh12345678): 1234567890123456
Введите IV (ровно 16 символов, например 12345678abcdefgh): 1234567890123456
Зашифровано. Результат: text.txt.cfb

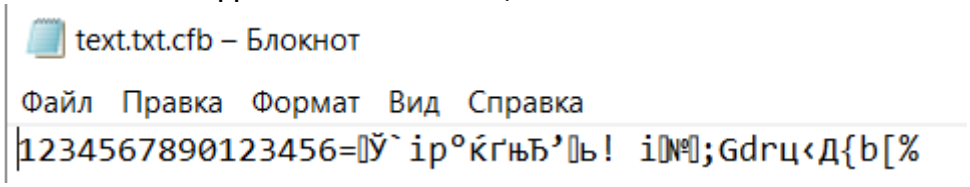
-Ваш выбор: 4
Имя зашифрованного файла (.cfb): text.txt.cfb
Введите ключ (ровно 16 символов, например abcdefgh12345678): 1234567890123456
Расшифровано. Результат: text.txt.cfb.decrypted.txt

-Ваш выбор: 4
Имя зашифрованного файла (.cfb): text.txt.cfb
Введите ключ (ровно 16 символов, например abcdefgh12345678): 1234567890123455
Ошибка: Неверный padding

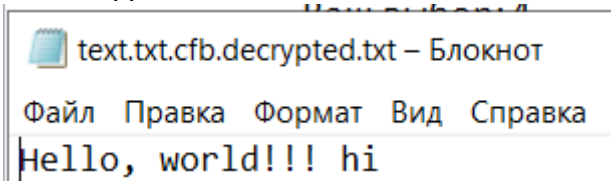
Исходное сообщение:



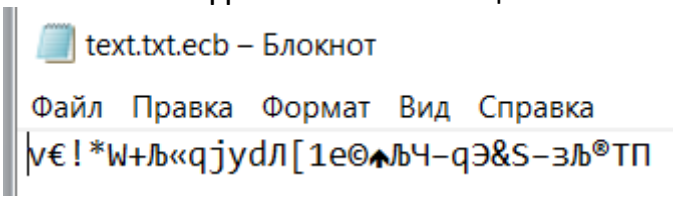
Файл с зашифрованным сообщением:



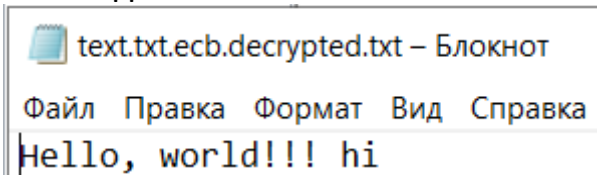
Расшифрованным:



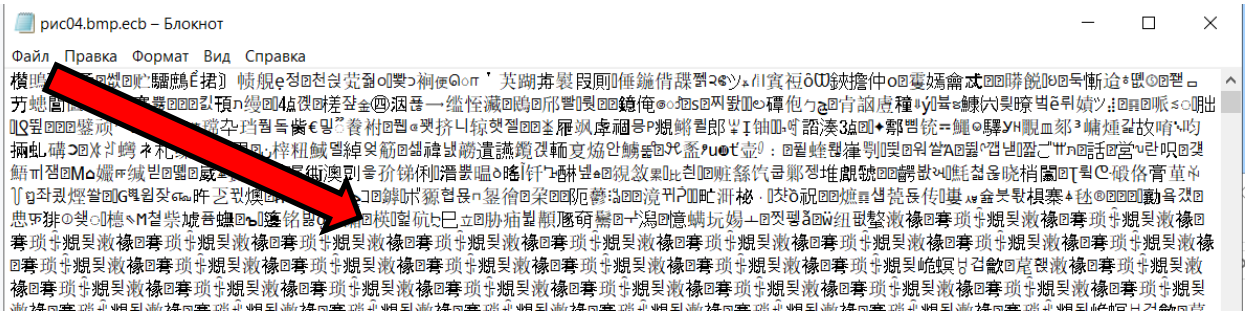
Файл с зашифрованным сообщением ECB:



Расшифрованным ECB:



При шифровании ECB битмап изображение, с большим наличием белого цвета получили выраженный паттерн:



Вывод : для защиты данных при передаче используется шифрование. Одним из классических способов считается шифрование с применением симметричного ключа, когда для шифрования и дешифрование используется один и тот же ключ. Минус такого способа в том, что при утрате ключом секретности, информацию легко расшифровать.